
ReSource PK Backend — Unified Specification

BACKEND TECHNICAL SPECIFICATION

1. Document Purpose

This document is the single backend reference for ReSource PK. It consolidates the project proposal, architecture notes, task plan, payment decisions, dashboard requirements, and blockchain plan into one implementation guide.

The backend supports a circular-economy marketplace connecting Pakistani textile factories and sellers with small businesses and buyers. The platform allows sellers to list leftover materials, buyers to discover and pool demand, and both parties to complete verifiable transactions.

This document is the source of truth for:

- Backend architecture
- User roles and permissions
- Database design
- REST API contracts
- Dashboard data requirements
- Group-buying and dynamic-pricing logic
- Rapid Gateway payment flow
- Polygon Amod transaction verification
- Security and failure handling

2. Confirmed Product Decisions

Area	Decision
Backend	Node.js with Express
Database	PostgreSQL
Authentication	JWT-based authentication with role-based access control
Roles	Buyer, Seller, Admin
Admin creation	Invite-only or controlled database seeding; no public Admin registration
Dashboard	One master dashboard layout with role-specific tabs and data
Payments	Rapid Gateway
Payment methods	Rapid Gateway-supported fiat methods such as JazzCash, Easypaisa, Raast, cards, and bank transfers, subject to enabled gateway configuration
Crypto payments	Excluded from the current MVP
Group-order payment	Authorized/held until the group target is reached
Blockchain	Polygon Amoy testnet for transaction passports and verification
AI search	Gemini integration with keyword-search fallback
Notifications	In-app notifications; SMS is optional and must not block core flows
Frontend	Separate frontend repository consuming documented backend APIs

3. Product Actors

Buyer

Buyers are small businesses, tailors, designers, students, and other users who need affordable quantities of leftover textile material.

Primary activities:

- Search and filter listings
- Use conversational search
- Post material requirements
- Join group orders
- Authorize payment for their contribution
- Track orders and payments

- Confirm receipt
- Rate sellers

Seller

Sellers are textile factories, mills, garment manufacturers, and other suppliers with reusable leftover material.

Primary activities:

- Create and manage listings
- Manage available inventory
- Define group-order minimums and price tiers
- Monitor buyers and orders
- Track revenue and profit/loss
- View ratings and reviews
- View blockchain records for completed transactions

Admin

Admins operate and moderate the platform.

Primary activities:

- Manage users and invitations
- Moderate listings
- Monitor orders and payments
- Resolve disputes
- Review platform analytics
- Monitor Rapid Gateway and blockchain health
- Review audit logs and suspicious activity

4. Backend Architecture

The backend is an Express REST API organized by responsibility rather than by a single large controller file.

Recommended structure:

```

src/
  app.js
  server.js
  config/
    env.js
    database.js
  routes/
    authRoutes.js
    userRoutes.js
    listingRoutes.js
    searchRoutes.js
    requirementRoutes.js
    groupOrderRoutes.js
    paymentRoutes.js
    orderRoutes.js
    ratingRoutes.js
    dashboardRoutes.js
    blockchainRoutes.js
    adminRoutes.js
    notificationRoutes.js
  controllers/
  services/
    authService.js
    listingService.js
    searchService.js
    matchingService.js
    groupOrderService.js
    pricingService.js
    paymentService.js
    rapidGatewayService.js
    orderService.js
    blockchainService.js
    dashboardService.js
    notificationService.js
  middleware/
    authMiddleware.js
    roleMiddleware.js
    validationMiddleware.js
    errorMiddleware.js
  models/
  validators/
  utils/
  db/
  migrations/
  seeds/

```

Architectural rules

1. Routes receive requests and delegate to controllers.
2. Controllers validate input and format HTTP responses.
3. Services contain business logic and integrations.
 1. Database operations must be parameterized and transaction-safe.
 2. External integrations must be isolated behind service adapters.
 3. Payment and blockchain failures must be logged and represented by explicit statuses.
 4. A non-critical notification failure must never cancel a successful order or payment flow.

5. API Conventions

Base URL:

```
/api/v1
```

Successful response format:

```
{
  "success": true,
  "data": {},
  "message": "Optional human-readable message"
}
```

Error response format:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "The request contains invalid fields.",
    "fields": {}
  }
}
```

Common HTTP statuses:

Status	Meaning
200	Successful read or action
201	Resource created
400	Invalid request
401	Missing or invalid authentication
403	Authenticated but not authorized
404	Resource not found
409	Conflict or invalid state transition
422	Validation failure
429	Rate limit exceeded
500	Unexpected server error
502	External provider failure

Pagination uses:

```
?page=1&limit=20&sort=created_at&order=desc
```

6. Authentication and Authorization

Registration

Public registration is allowed for Buyer and Seller accounts only.

```
POST /api/v1/auth/register
```

Request:

```
{
  "name": "Ayesha Khan",
  "email": "ayesha@example.com",
  "password": "strong-password",
  "phone": "+923001234567",
  "city": "Karachi",
  "role": "BUYER"
}
```

Allowed public roles:

```
BUYER, SELLER
```

The API must reject public attempts to register as Admin.

Login

```
POST /api/v1/auth/login
```

Returns a JWT access token and the authenticated user profile.

Admin invitations

```
POST /api/v1/admin/invitations
GET /api/v1/admin/invitations
POST /api/v1/admin/invitations/:token/accept
```

Only an existing Admin can create an Admin invitation. Invitation tokens must be single-use, expiring, hashed at rest, and invalidated after acceptance.

Authentication header

```
Authorization: Bearer <jwt>
```

JWT claims should include:

```
{
  "sub": "user-id",
  "role": "BUYER",
  "email": "user@example.com",
  "iat": 0,
  "exp": 0
}
```

Role protection

Use two middleware layers:

- `protect`: validates the JWT and loads the user.
- `requireRole('BUYER')`, `requireRole('SELLER')`, or `requireRole('ADMIN')`: restricts access by role.

Ownership checks are also required. For example, a Seller may update only their own listing, and a Buyer may view their own payment records.

7. Database Model

The minimum PostgreSQL schema should include the following tables.

users

- `id` UUID primary key
- `name`
- `email` unique
- `password_hash`
- `phone`
- `city`
- `role` enum: `BUYER`, `SELLER`, `ADMIN`
- `status` enum: `ACTIVE`, `SUSPENDED`, `PENDING`
- `created_at`
- `updated_at`

admin_invitations

- `id`
- `email`
- `token_hash`
- `invited_by`
- `expires_at`
- `accepted_at`
- `created_at`

listings

- `id`
- `seller_id`
- `title`
- `description`
- `material_type`
- `color`
- `condition`
- `quantity_available_kg`
- `minimum_group_quantity_kg`
- `location`
- `photos`
- `status`: `DRAFT`, `ACTIVE`, `PAUSED`, `SOLD_OUT`, `REMOVED`
- `created_at`
- `updated_at`

listing_price_tiers

- `id`
- `listing_id`
- `minimum_quantity_kg`
- `price_per_kg_pkr`

requirements

- `id`
- `buyer_id`
- `material_type`
- `description`

- `minimum_quantity_kg`
- `maximum_quantity_kg`
- `maximum_price_per_kg_pkr`
- `location`
- `status`: `OPEN`, `MATCHED`, `CLOSED`
- `created_at`
- `updated_at`

group_orders

- `id`
- `listing_id`
- `seller_id`
- `target_quantity_kg`
- `pooled_quantity_kg`
- `current_price_per_kg_pkr`
- `status`: `OPEN`, `TARGET_REACHED`, `PAYMENT_CAPTURE_PENDING`, `CONFIRMED`, `COMPLETED`, `EXPIRED`, `CANCELLED`
- `deadline_at`
- `confirmed_at`
- `completed_at`
- `created_at`
- `updated_at`

When a group order is created, it must snapshot the relevant listing and price-tier data. Later listing edits must not change an active group order.

group_order_contributions

- `id`
- `group_order_id`
- `buyer_id`
- `quantity_kg`
- `price_per_kg_pkr`
- `amount_pkr`
- `payment_status`: `NOT_STARTED`, `AUTHORIZED`, `CAPTURED`, `RELEASED`, `REFUNDED`, `FAILED`
- `payment_id`
- `created_at`
- `updated_at`

orders

- `id`
- `group_order_id`
- `listing_id`
- `seller_id`
- `buyer_id`
- `quantity_kg`
- `price_per_kg_pkr`
- `total_amount_pkr`
- `status`: `PENDING`, `CONFIRMED`, `READY_FOR_PICKUP`, `RECEIVED`, `COMPLETED`, `DISPUTED`, `CANCELLED`
- `received_at`
- `completed_at`
- `created_at`
- `updated_at`

payments

- `id`
- `user_id`
- `group_order_id`
- `contribution_id`
- `provider`: `RAPID_GATEWAY`
- `provider_transaction_id`
- `method`
- `amount_pkr`
- `status`: `CREATED`, `PENDING`, `AUTHORIZED`, `CAPTURED`, `RELEASED`, `REFUNDED`, `FAILED`
- `provider_payload` JSONB
- `failure_reason`
- `authorized_at`
- `captured_at`
- `released_at`
- `created_at`
- `updated_at`

ratings

- `id`
- `order_id`
- `rater_id`
- `ratee_id`
- `stars`
- `comment`
- `created_at`

Add a unique constraint on `(order_id, rater_id)`.

blockchain_records

- `id`
- `order_id`
- `hash`
- `network`: `POLYGON_AMOY`
- `contract_address`
- `transaction_hash`
- `status`: `PENDING`, `CONFIRMED`, `FAILED`, `SIMULATED`
- `failure_reason`
- `created_at`
- `confirmed_at`

notifications

- `id`
- `user_id`
- `type`
- `title`
- `message`
- `read_at`
- `created_at`

disputes

- `id`
- `order_id`
- `opened_by`
- `reason`

- `description`
- `status`: `OPEN`, `UNDER_REVIEW`, `RESOLVED`, `REJECTED`
- `resolution`
- `resolved_by`
- `resolved_at`
- `created_at`

audit_logs

- `id`
- `actor_id`
- `action`
- `resource_type`
- `resource_id`
- `metadata` JSONB
- `ip_address`
- `created_at`

8. Listing and Search APIs

```
POST /api/v1/listings
GET /api/v1/listings
GET /api/v1/listings/:id
PATCH /api/v1/listings/:id
DELETE /api/v1/listings/:id
POST /api/v1/listings/:id/pause
POST /api/v1/listings/:id/photos
```

Seller-only operations must enforce ownership.

Structured search:

```
GET /api/v1/search?
material_type=denim&color=blue&min_quantity=5&max_price=300&location=Karachi
```

Conversational search:

```
POST /api/v1/search/natural
```

Request:

```
{
  "query": "I need 10kg of durable blue denim near Karachi under PKR 300 per kg"
}
```

The Gemini response should be converted into validated filters. If Gemini fails or times out, the service must fall back to keyword and structured SQL search.

Requirement APIs:

```
POST /api/v1/requirements
GET /api/v1/requirements/me
GET /api/v1/requirements/:id
PATCH /api/v1/requirements/:id
DELETE /api/v1/requirements/:id
GET /api/v1/requirements/:id/matches
```

9. Group Buying and Dynamic Pricing

Create a group order:

```
POST /api/v1/group-orders
```

Join a group order:

```
POST /api/v1/group-orders/:id/join
```

View group order:

```
GET /api/v1/group-orders/:id
```

View current user contributions:

```
GET /api/v1/group-orders/:id/contributions/me
```

Required transaction behavior

Joining a group order must happen inside a database transaction with row-level locking:

1. Lock the group-order row.
2. Confirm the group order is still open.
3. Confirm the requested quantity does not exceed the remaining quantity.
4. Add the contribution.
5. Recalculate pooled quantity and price tier.
6. Determine whether the target has been reached.
7. Commit the transaction.

The system must never use a separate read-then-write sequence for pooled quantity updates.

When the target is reached:

1. Mark the group order as `TARGET_REACHED`.
2. Stop new contributions.
3. Capture authorized payments.
4. Create or confirm the related orders.
5. Mark the group order `CONFIRMED` after successful payment processing.
6. Notify participants.
7. Start the completion and blockchain flow after delivery.

10. Rapid Gateway Payment Integration

All payment provider logic must be isolated behind `rapidGatewayService.js` or an equivalent adapter. The rest of the application must not depend directly on provider-specific request formats.

Suggested service methods:

```
createPaymentIntent(data)
authorizePayment(data)
capturePayment(providerPaymentId)
releasePayment(providerPaymentId)
refundPayment(providerPaymentId, amount)
verifyCallback(payload, signature)
getPaymentStatus(providerPaymentId)
```

Payment APIs:

```
POST /api/v1/payments/authorize
GET /api/v1/payments/:id
POST /api/v1/payments/:id/capture
POST /api/v1/payments/:id/release
POST /api/v1/payments/:id/refund
POST /api/v1/payments/rapid-gateway/callback
```

Payment flow

1. Buyer joins a group order.
2. Backend calculates the buyer's quantity and contribution amount.
3. Backend creates a Rapid Gateway payment request.
1. Buyer completes the gateway authorization step.
2. Backend verifies the provider callback/signature.
3. Payment is stored as `AUTHORIZED`.
4. Group order remains open until the target is reached.
5. When the target is reached, authorized payments are captured.

6. If the group order expires or is cancelled, authorized payments are released or refunded according to Rapid Gateway capabilities.

The exact provider callback fields, signature rules, enabled methods, and capture/release behavior must be implemented from the official Rapid Gateway merchant documentation and sandbox credentials. These details must not be guessed or hardcoded from informal messages.

The MVP does not include cryptocurrency or USDT payment processing.

11. Order Lifecycle

```
PENDING
-> CONFIRMED
-> READY_FOR_PICKUP
-> RECEIVED
-> COMPLETED
```

Alternative paths:

```
PENDING -> CANCELLED
CONFIRMED -> DISPUTED
PENDING -> DISPUTED
```

Only the Buyer can mark an order as received. A completed order can trigger ratings and blockchain hashing.

12. Blockchain Material Passport

Blockchain is used for verification of completed transactions, not for payment settlement.

When an order is completed, the backend creates a deterministic hash from stable transaction data, for example:

```
order_id
listing_id
seller_id
buyer_id
quantity_kg
price_per_kg_pkr
completed_at
```

The original sensitive user data must not be written directly to the public chain.

APIs:

```
POST /api/v1/orders/:id/blockchain-record
GET  /api/v1/orders/:id/verify
GET  /api/v1/blockchain/health
```

The blockchain service should:

1. Generate the deterministic hash.
2. Store a local `PENDING` record.
3. Submit the hash to the Polygon Amoy contract.
4. Store the transaction hash.
5. Confirm the transaction when mined.
6. Return a verifiable match or no-match result.

If Polygon or the RPC provider fails, the completed order must remain completed. The failure is recorded and can be retried. For a demo fallback, the local hash may be marked `SIMULATED`, but it must never be presented as a confirmed on-chain record.

13. Role-Based Master Dashboard APIs

The frontend will use one dashboard layout. The backend returns the tabs and data allowed for the authenticated role.

```
GET /api/v1/dashboard
```

Example response shape:

```
{
  "success": true,
  "data": {
    "role": "SELLER",
    "tabs": [
      "overview",
      "listings",
      "inventory",
      "group_orders",
      "sales",
      "analytics",
      "ratings",
      "blockchain"
    ],
    "overview": {},
    "metrics": {},
    "recent_activity": []
  }
}
```

Optional detailed endpoints:

```
GET /api/v1/dashboard/overview
GET /api/v1/dashboard/analytics
GET /api/v1/dashboard/recent-activity
```

Buyer dashboard data

- Search shortcut and recommended listings
- Open requirements and matched listings
- Active group orders
- Contribution quantities and statuses
- Payment authorization/capture/release status
- Completed purchases
- Order progress
- Ratings submitted and pending ratings
- Notifications

Seller dashboard data

- Active, paused, and sold-out listings
- Inventory quantity and utilization
- Active group orders and progress
- Confirmed and completed sales
- Revenue totals
- Profit/loss summary
- Average selling price
- Material demand trends
- Sales by material type and location
- Order conversion trends
- Buyer ratings and review summary
- Blockchain records
- Environmental impact metrics

Profit/loss requires a defined seller cost basis. The backend should store or calculate costs using a future `listing_cost_per_kg_pkr` or seller-entered transaction cost field. Until that field is approved, the dashboard must label the figure as revenue or estimated profit rather than true profit.

Admin dashboard data

- Total users by role and status
- New users and active users
- Listings by status
- Group orders by status
- Orders and transaction volume
- Payment totals and failed payments

- Open disputes
- Platform revenue or fees, if applicable
- Total material listed, sold, and diverted from waste
- Blockchain success and failure counts
- Rapid Gateway health and callback failures
- Suspicious activity and audit events

14. Ratings and Reviews

```
POST /api/v1/orders/:id/ratings
GET  /api/v1/users/:id/ratings
```

Rules:

- Ratings are allowed only for completed orders.
- Buyer rates Seller and Seller rates Buyer.
- Each user may rate the other party only once per order.
- Stars must be between 1 and 5.
- Users cannot rate themselves.

15. Admin and Moderation APIs

```
GET  /api/v1/admin/users
PATCH /api/v1/admin/users/:id/status
GET  /api/v1/admin/listings
PATCH /api/v1/admin/listings/:id/status
GET  /api/v1/admin/orders
GET  /api/v1/admin/payments
GET  /api/v1/admin/disputes
PATCH /api/v1/admin/disputes/:id
GET  /api/v1/admin/metrics
GET  /api/v1/admin/audit-logs
```

All Admin actions must generate audit-log entries.

16. Disputes

```
POST /api/v1/orders/:id/disputes
GET  /api/v1/disputes/me
GET  /api/v1/admin/disputes
PATCH /api/v1/admin/disputes/:id
```

Opening a dispute must not automatically reverse a payment. An Admin must review the case and explicitly apply the approved resolution.

17. Security Requirements

- Hash passwords using Argon2 or bcrypt.
- Never store plaintext passwords or raw payment secrets.
- Store Rapid Gateway credentials only in environment variables or a secrets manager.
- Verify payment callbacks and reject duplicate callbacks.
- Use idempotency keys for payment authorization, capture, release, and refund operations.
- Validate all request bodies, query parameters, and route parameters.
- Use parameterized SQL queries or a safe query builder/ORM.
- Apply rate limiting to authentication, payment, and callback endpoints.
- Restrict CORS to the deployed frontend origin in production.
- Avoid exposing provider payloads or sensitive data in normal API responses.
- Record security-sensitive actions in audit logs.
- Use database transactions for group-order and payment state changes.

18. Environment Variables

```

NODE_ENV=development
PORT=5000
DATABASE_URL=
JWT_SECRET=
JWT_EXPIRES_IN=7d
FRONTEND_URL=

RAPID_GATEWAY_BASE_URL=
RAPID_GATEWAY_API_KEY=
RAPID_GATEWAY_MERCHANT_ID=
RAPID_GATEWAY_SECRET=
RAPID_GATEWAY_CALLBACK_SECRET=

GEMINI_API_KEY=

POLYGON_AMOY_RPC_URL=
POLYGON_PRIVATE_KEY=
POLYGON_CONTRACT_ADDRESS=

TWILIO_ACCOUNT_SID=
TWILIO_AUTH_TOKEN=
TWILIO_FROM_NUMBER=

```

Crypto-specific environment variables must not be added to the MVP configuration because crypto payments are out of scope.

19. Failure and Concurrency Rules

Group-order concurrency

Use row locks or atomic updates when multiple buyers join the same group order. No buyer contribution may overwrite another contribution.

Listing snapshot

An active group order uses its own snapshot of listing quantity, price tiers, material details, and target quantity.

Expiration

Every group order should have a deadline. On expiration, the system must stop new contributions and release or refund authorized payments according to the gateway's supported operation.

Gemini failure

Fall back to structured and keyword SQL search.

Rapid Gateway failure

Store the payment as failed or pending, return a clear status to the frontend, and do not mark the order as confirmed until payment state is verified.

Blockchain failure

Do not block order completion. Store the local hash and failed blockchain status, then retry asynchronously.

Notification failure

Do not roll back a successful order or payment because SMS, email, or in-app notification delivery failed.

20. Implementation Order

Phase 1 — Foundation

- Configure Express, environment validation, CORS, and error handling.
- Connect PostgreSQL.
- Create migrations and seed data.

- Implement user registration and login.
- Implement JWT protection and role middleware.
- Implement Admin invitation flow.

Phase 2 — Marketplace

- Implement listings CRUD.
- Implement photo upload integration.
- Implement structured search and filters.
- Implement conversational search with keyword fallback.
- Implement buyer requirements and matching.

Phase 3 — Group Buying and Payments

- Implement listing price tiers.
- Implement group orders with atomic locking.
- Implement contribution tracking.
- Implement Rapid Gateway authorization callbacks.
- Implement payment capture, release, and refund handling.
- Implement group-order expiration.

Phase 4 — Orders and Trust

- Implement order state transitions.
- Implement delivery/received flow.
- Implement ratings and reviews.
- Implement Polygon Amoy hash creation and verification.
- Implement notifications.

Phase 5 — Dashboards and Admin

- Implement role-specific dashboard aggregation.
- Implement Seller profit/loss and commerce analytics.
- Implement Admin metrics and moderation APIs.
- Implement disputes and audit logs.
- Add integration and end-to-end tests.

21. Required Testing Scenarios

- Buyer registration and login
- Seller registration and login
- Public attempt to register as Admin is rejected
- Admin invitation acceptance
- Expired invitation is rejected
- Seller can create and update their own listing
- Seller cannot update another Seller's listing
- Buyer can search listings
- Gemini failure falls back to keyword search
- Two buyers joining simultaneously update pooled quantity correctly
- Group order reaches target exactly once
- Payment callback is idempotent
- Authorized payments are captured after target completion
- Authorized payments are released/refunded after expiration
- Failed gateway payment does not confirm an order
- Buyer and Seller can complete an order lifecycle
- Only completed orders can be rated
- Completed order generates a blockchain verification record
- Blockchain failure does not block order completion
- Buyer cannot access Seller or Admin dashboard data
- Seller cannot access Admin dashboard data
- Admin actions create audit logs

22. Open Implementation Dependencies

The following information must be confirmed from the official provider and deployment accounts before implementation:

1. Rapid Gateway sandbox base URL and authentication method.
2. Supported sandbox payment methods and exact merchant onboarding requirements.
3. Authorization, capture, release, and refund capabilities.
4. Callback URL format, signature verification, and retry behavior.
5. Rapid Gateway idempotency requirements.
6. Frontend repository location and expected API base URL.
7. PostgreSQL hosting choice, such as Supabase or Neon.
8. Polygon Amoy RPC provider and deployed contract details.

9. Seller cost fields required for accurate profit/loss reporting.
10. Whether the platform charges a commission or service fee.

Until these dependencies are confirmed, provider-specific fields must remain inside adapter services and must not leak into the general business logic.